

Guia Rápido: Introdução ao LangGraph

Conceito Central

LangGraph = StateGraph onde cada nó é uma função Python e cada aresta é uma transição.

State (dict) → Node (função) → State atualizado → Node → ... → END

1. Primeiro Grafo: Eco

```
# 07_primeiro_grafo.py
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    mensagem: str
    status: str

def entrada(state: State) -> dict:
    """Nó que processa a entrada"""
    return {"status": "processado"}

def saida(state: State) -> dict:
    """Nó que prepara a saída"""
    return {"mensagem": f"Eco: {state['mensagem']}"}

# Montar grafo
grafo = StateGraph(State)
grafo.add_node("entrada", entrada)
grafo.add_node("saida", saida)
grafo.add_edge(START, "entrada")
grafo.add_edge("entrada", "saida")
grafo.add_edge("saida", END)

app = grafo.compile()

# Executar
resultado = app.invoke({"mensagem": "Olá LangGraph!"})
print(resultado)
# {'mensagem': 'Eco: Olá LangGraph!', 'status': 'processado'}
```

2. Conditional Edges (Roteamento)

```
# 08_rotas.py
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    texto: str
    sentimento: str
    resposta: str

def analisar_sentimento(state: State) -> dict:
    texto = state["texto"].lower()
    if any(p in texto for p in ["ótimo", "bom", "amo", "👍"]):
        return {"sentimento": "positivo"}
    elif any(p in texto for p in ["ruim", "péssimo", "odeio"]):
        return {"sentimento": "negativo"}
    return {"sentimento": "neutro"}

def responder_positivo(state: State) -> dict:
    return {"resposta": "Que bom! Fico feliz em ajudar 😊"}

def responder_negativo(state: State) -> dict:
    return {"resposta": "Sinto muito. Como posso melhorar?"}

def responder_neutro(state: State) -> dict:
    return {"resposta": "Entendi. Conte-me mais."}

def rotear(state: State) -> str:
    return state["sentimento"] # "positivo" | "negativo" | "neutro"

grafo = StateGraph(State)
grafo.add_node("analisar", analisar_sentimento)
grafo.add_node("positivo", responder_positivo)
grafo.add_node("negativo", responder_negativo)
grafo.add_node("neutro", responder_neutro)

grafo.add_edge(START, "analisar")
grafo.add_conditional_edges(
    "analisar",
    rotear,
    {
        "positivo": "positivo",
        "negativo": "negativo",
        "neutro": "neutro",
    },
)
grafo.add_edge("positivo", END)
grafo.add_edge("negativo", END)
grafo.add_edge("neutro", END)
```

```

app = grafo.compile()

print(app.invoke({"texto": "Que produto incrível!"}))
print(app.invoke({"texto": "Não gostei do atendimento"}))
print(app.invoke({"texto": "Qual o horário de funcionamento?"}))

```

3. Streaming (Eventos em Tempo Real)

```

# 09_streaming.py
import asyncio
from langgraph.graph import StateGraph, START, END

# Mesmo grafo do exemplo 2...
# (reuse a definição do grafo acima)

async def main():
    app = grafo.compile()

    async for event in app.astream(
        {"texto": "Que produto incrível!"},
        stream_mode="updates",
    ):
        for node_name, update in event.items():
            print(f"[{node_name}] → {update}")

asyncio.run(main())
# [analisar] → {'sentimento': 'positivo'}
# [positivo] → {'resposta': 'Que bom! Fico feliz em ajudar 😊'}

```

4. Checkpoints (Persistência)

```

# 10_checkpoints.py
from langgraph.checkpoint.memory import MemorySaver

# Ao compilar:
app = grafo.compile(checkpointer=MemorySaver())

config = {"configurable": {"thread_id": "conversa-123"}}

# Cada invoke com mesmo thread_id compartilha estado
r1 = app.invoke({"texto": "Meu nome é Rodolfo"}, config)
r2 = app.invoke({"texto": "Qual meu nome?"}, config)

# Histórico de checkpoints
estados = list(app.get_state_history(config))
for s in estados:
    print(s.values)

```

5. Send API (Fan-Out Paralelo)

```
# 11_fanout.py
from langgraph.types import Send

AGENTES = ["copywriter", "designer", "estrategista"]

def disparar_agentes(state: State) -> list[Send]:
    """Retorna Send para cada agente executar em paralelo"""
    return [
        Send("executar", {"agente": nome, "tarefa":
            state["texto"]})
        for nome in AGENTES
    ]

async def executar(state: State) -> dict:
    resultado = f"[{state['agente']}] executou: {state['tarefa']}
        [:50]}"
    return {"resultados": [resultado]}

grafo = StateGraph(State)
grafo.add_node("disparar", disparar_agentes)
grafo.add_node("executar", executar)
grafo.add_edge(START, "disparar")
grafo.add_edge("executar", END)

# Conditional edge do fanout:
# grafo.add_conditional_edges("disparar", disparar_agentes,
#                             path_map)
```

Fluxo Completo: Classificar → Responder

```
# 12_completo.py
from typing import TypedDict, Annotated
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages

class AgentState(TypedDict):
    messages: Annotated[list, add_messages]
    route: str
    response: str

def classify(state: AgentState) -> dict:
    """Classifica a intenção"""
    msg = state["messages"][-1].content.lower()
    if "?" in msg:
        return {"route": "pergunta"}
    elif len(msg) > 200:
        return {"route": "analise"}
    return {"route": "rapido"}
```

```

def responder_pergunta(state: AgentState) -> dict:
    return {"response": "Resposta detalhada para sua pergunta..."}

def responder_analise(state: AgentState) -> dict:
    return {"response": "Análise profunda do seu texto..."}

def responder_rapido(state: AgentState) -> dict:
    return {"response": "Resposta rápida e direta."}

def rotear(state: AgentState) -> str:
    return state["route"]

grafo = StateGraph(AgentState)
grafo.add_node("classify", classify)
grafo.add_node("pergunta", responder_pergunta)
grafo.add_node("analise", responder_analise)
grafo.add_node("rapido", responder_rapido)

grafo.add_edge(START, "classify")
grafo.add_conditional_edges(
    "classify", rotear,
    {"pergunta": "pergunta", "analise": "analise", "rapido":
     "rapido"},
)
grafo.add_edge("pergunta", END)
grafo.add_edge("analise", END)
grafo.add_edge("rapido", END)

app = grafo.compile()

# Com mensagens LangChain
from langchain_core.messages import HumanMessage
result = app.invoke({"messages": [HumanMessage(content="O que é
    LangGraph?")]}))
print(result["response"])

```

Glossário Rápido

Termo	Significado
State	Dict tipado que carrega dados entre nós
Node	Função Python que processa o state
Edge	Conexão fixa entre dois nós
Conditional Edge	Roteamento dinâmico baseado no state
Send	Dispara múltiplas execuções paralelas
Checkpoint	Persiste o state após cada nó
Thread	Sessão isolada de conversa (thread_id)

Próximo: Volte ao [README do projeto](#) e execute o Mega Brain v2!